Bruce Xi

Greater Toronto Area, ON, Canada
(236)-777-8218 • brucexi99@outlook.com • bruce-shidi-xi
brucexi999

Experience

Huawei Canada — **Markham, ON, Canada**
SW/HW Co-Design Engineer — *Jan. 2025 – Present*

Trace ML training failures and performance bottlenecks to their root causes across agent workflows, runtimes, kernels, and compiler IR; implement fixes and validate their impact on Ascend NPUs.

Post-Training for a Triton Kernel Agent May 2026 – Oct. 2026

ATRL (ICLR 2027 submission) trains an agent to turn PyTorch operators into optimized Triton kernels on Ascend NPUs. The team built a custom Triton kernel agent harness and introduced compaction-aware RL. Compared with untuned Qwen3.6-35B, ATRL raised KernelBench L1 Pass@1 from **20% to 73%** and Fast@1 (correct kernels at least as fast as eager PyTorch) from **8% to 23%**. Recipe: verl-ascend-recipe PR #49.

- Implemented partial rollout in verl/vLLM to reduce waiting on long-running agent trajectories, accelerating RL training by **25%**; added cross-batch scheduling, resumable execution, and importance-sampling correction across policy updates.
- Distilled and curated execution-verified SFT trajectories; filtered examples by correctness and speedup, pruned unproductive turns, and compressed reasoning to fit the training token budget.
- Manually inspected agent trajectories turn by turn to debug the training system, identifying four issues across prompts, skills, and tool interfaces and proposing fixes.
- Integrated a profiler and Ascend-specific optimization skills into the Triton kernel agent's harness for RL training; one agent-generated matmul implementation achieved **24% lower latency** (2,886 to 2,197 μs).
- Diagnosed convolution verification failures caused by HF32 reference execution instead of full FP32, correcting an evaluation issue initially attributed to the agent's kernel-generation ability.
- Ran controlled ablations of agent harnesses and Ascend-specific skills to inform the RL training setup.

Triton-to-NPU Compiler Debugging (LLVM/MLIR) Feb. 2026 – Apr. 2026

MLIR-based compiler middle-end that lowers Triton programs to LLVM IR for Ascend NPUs.

- Resolved silent data corruption in Triton-to-NPU compilation by tracing failures through **600+ compiler pass dumps** to incorrect hardware-predicate materialization in MLIR-to-LLVM lowering.
- Used intermediate-value logging and SSA manipulation to isolate corrupted vector lanes; implemented the fix and regression tests (AscendNPU-IR MR #712).

Training Performance Optimization for a Mamba2 Model Mar. 2025 – Dec. 2025

Custom Ascend NPU kernels to reduce memory traffic during pre-training of a Mamba2-based model.

- Integrated fused Ascend C kernels into PyTorch via Pybind; one kernel achieved **12× forward and 15× backward speedups**, reducing training-step time by **4.6%** and improving MFU by **1.5%**.
- Optimized tiling, workload partitioning, memory access, and compute/data-transfer overlap; validated numerical correctness and diagnosed synchronization faults and pipeline bubbles with Ascend's profiler, debugger, and simulator.

Education

The University of British Columbia — **Vancouver, BC, Canada**
M.Eng. in Electrical and Computer Engineering, GPA: 4.0 — *2021 – 2024*

Imperial College London — **London, UK**
B.Eng. in Materials Science and Engineering, First-Class Honours — *2018 – 2021*
Dean's List for three consecutive years (2018–2021)

Skills

Languages & frameworks: Python, C/C++, PyTorch, Triton, Ascend C

ML systems: verl, vLLM, MindSpeed-LLM; SFT/RL, GRPO, rollout scheduling, training parallelism

Performance: Profiling, benchmarking, roofline analysis, numerical validation; LLVM/MLIR debugging